

Day04.5 学习文档 v1.0 : Agent Runtime Industrial Mapping (工业术语映射)

本文是《从零实现 Agent Runtime》学习阶段的 Day04.5 正式学习文档。

Day04.5 是一个临时插入但非常必要的桥梁章节：原计划在 Day04 Part E 之后直接进入 Day05 Tool Calling，但在进入 Day05 时发现，Day01-Day04 已经建立了自己的 Runtime 工程认知，却还没有系统补齐这些概念在工业 Agent 框架里的常用术语、源码命名和面试表达。因此 Day05 暂时后移，先补上 Day04.5。

本节定位

Day01-Day04 的学习主线是：

如何从零设计一个 Agent Runtime？

Day04.5 要解决的是：

这些 Runtime 概念在工业 Agent 框架里通常叫什么？

这一节不引入新的 Runtime 能力，而是做一次 Terminology Mapping Pass：

```
Mini Runtime Concept
|
v
Industry Terminology
|
v
Framework Implementation
|
v
Interview Expression
```

也就是说，Day04.5 是从“自研 Runtime 视角”进入“工业 Agent 视角”的知识迁移层。

为什么需要 Day04.5

Day01-Day04 已经建立了底层认知：

- Agent 不是一次 LLM 调用，而是一个持续运行系统
- Runtime 是 Agent 的运行环境和控制中心
- Runtime State 描述 Agent 当前世界
- Context Builder 将 Runtime State 投影成 LLM 可见上下文
- Context Window Management 管理有限 token 空间
- Provider Adapter 隔离模型供应商协议差异

但真实工业框架往往不会完全使用这些命名。比如：

- Runtime Loop 可能被称为 Agent Loop、Execution Loop、ReAct Loop
- Runtime State 可能被称为 Agent State、Workflow State、StateGraph
- Context Builder 可能被称为 Context Engineering Pipeline
- Provider Adapter 可能被称为 LLM Gateway、Model Router、Model Abstraction Layer

- Tool Result 在 ReAct 语境下通常被称为 Observation

如果不补这层映射，后面阅读 OpenAI Agents SDK、LangGraph、Claude Code、CrewAI、Mastra、AutoGen 或 MCP 时，会误以为它们是完全不同的东西。

Day04.5 的目标是让我们看到：

大多数 Agent 框架，本质上都是 Agent Runtime 的不同实现方式。

Part A : Agent 基础概念映射

Day01 的核心认知是：

Agent = LLM + Runtime + Tools + Memory

工业界常见叫法包括：

AI Agent
Autonomous Agent
LLM-based Agent

更标准的表达是：

Agent is a system that enables an LLM to perceive, reason, act, and maintain state through an execution loop.

拆开来看：

Perception
感知

Reasoning
推理

Action
行动

State
状态

Feedback Loop
反馈循环

对应关系：

| 我们的概念 | 工业术语 | | Agent | AI Agent / Autonomous Agent / LLM-based Agent | | Runtime | Agent Runtime | | Tool | Tool Calling / Function Calling | | Memory | Agent Memory | | Planning | Task Planning | | Loop | Agent Loop |

面试表达：

An Agent is not just a single LLM call. It is a runtime system that enables an LLM to reason, act, observe results, and maintain state across multiple steps.

Part B : Runtime 核心架构映射

Day02-Day03 的核心认知是：

Runtime 是 Agent 的大脑

工业界常见叫法包括：

Agent Runtime

Agent Orchestrator
Agent Execution Engine
Agent Framework Core

不同框架里可能有不同名字：

| 框架 | 近似概念 | | | | OpenAI Agents SDK | Runner | | LangGraph | Graph Runtime | | Mastra | Agent Runtime | | CrewAI | Crew / Flow Runtime | | AutoGen | Agent Runtime / Conversation Runtime |

核心职责基本一致：

State Management
Context Construction
LLM Invocation
Tool Execution
Lifecycle Management
Observability

Day03 的 Runtime Core 可以映射为：

| Day03 工程概念 | 工业术语 | | | | Runtime Core | Agent Orchestrator | | State Manager | State Store / Agent State Manager | | Context Builder | Context Engineering Pipeline | | Tool Manager | Tool Runtime | | Memory Manager | Memory System | | Provider Adapter | Model Abstraction Layer / LLM Gateway | | Event Bus | Runtime Event System |

面试表达：

An Agent Runtime orchestrates state, context construction, model invocation, tool execution, persistence, and observability. It is the control plane of an agent system.

Part C：Runtime Loop 与 ReAct 映射

这是 Day04.5 最重要的补充之一。

我们之前说：

Runtime Loop

工业界常见叫法：

Agent Loop
Execution Loop
Reasoning Loop
ReAct Loop

它们之间的关系：

Runtime Loop
|
v
Agent Loop
|
v
ReAct
Reason + Act

经典 ReAct 模式：

Thought
思考

Action
行动

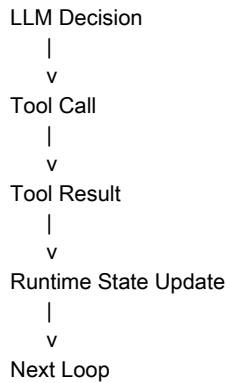
Observation

观察

Thought
思考

Action
行动

对应我们的 Runtime 语言：



所以后续看到这些词时，应该把它们放回 Runtime Loop：

| 工业术语 | Runtime 视角 | | Thought | LLM reasoning / decision | | Action | Tool call intent | | Observation | Tool result | | State Transition | Runtime state update | | Agent Loop | Runtime loop |

面试表达：

ReAct is a reasoning-and-acting pattern. In a runtime implementation, it is realized as an agent loop where the model produces actions, the runtime executes tools, observations are written back into state, and the next context is rebuilt.

Part D : Runtime State 映射

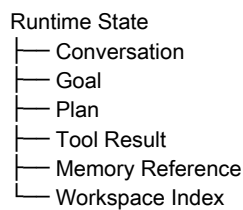
Day04 Part B 的核心是：

Runtime State 是 Agent 当前世界的结构化表示

工业界常见叫法：

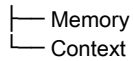
Agent State
Workflow State
Task State
State Machine
State Store

我们的 Runtime State：



工业 Agent State：

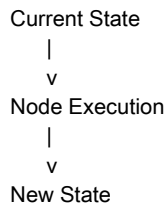
Agent State
| Messages
| Task State
| Workflow State
| Tool State



LangGraph 的核心概念 StateGraph，可以理解为：

State
+
Transition
=
StateGraph

一次节点执行就是：



对应 Day04 学过的 Runtime State Lifecycle：

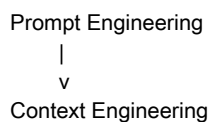
Create
Restore
Update
Persist
Checkpoint
Release

面试表达：

Runtime State is the agent's structured representation of the current world. Frameworks such as LangGraph expose this idea as graph state plus state transitions.

Part E：Context Engineering 映射

Day04 最大的认知升级是：



我们说的：

Context Builder

工业界通常会归入：

Context Engineering Pipeline
Context Management
Context Optimization
Prompt Assembly

完整映射：

| 我们的概念 | 工业术语 | | Retrieval | Retrieval | | Ranking | Relevance Ranking | | Compression | Context Compression | | Eviction | Context Pruning | | Assembly | Prompt Assembly / Message Assembly | | Token Budget | Context Budget Management | | Context Window Management | Context Optimization |

以前很多应用的核心能力是：

写一个好 Prompt

现代 Agent Runtime 的核心能力是：

设计一个动态 Context Pipeline

例如 Claude Code 不只是把用户问题加到系统 Prompt 后面，而是会组织：

```
用户任务
代码上下文
文件结构
Git 信息
历史修改
工具结果
当前计划
  |
  v
Context Assembly
  |
  v
LLM
```

这就是 Context Engineering。

面试表达：

Modern agent systems do not simply prompt LLMs. They perform context engineering by dynamically selecting, compressing, ranking, and assembling relevant information from runtime state under a token budget.

Part F : Memory 体系映射

Day04 中我们已经接触到：

```
Memory Reference
Persistent State
Summary Memory
```

工业界常见叫法：

```
Agent Memory System
Memory Store
Long-term Memory
Working Memory
```

常见分类：

```
Memory
├── Short-term Memory
├── Working Memory
├── Long-term Memory
├── Episodic Memory
├── Semantic Memory
└── Procedural Memory
```

对应关系：

| 工业术语 | Runtime 视角 | | - | | Short-term Memory | 最近几轮 Conversation History | | Working Memory | 当前 Runtime State / task state | | Long-term Memory | 跨 session 的用户、项目、知识记忆 | | Episodic Memory | 事件记忆，保存发生过的经验 | | Semantic Memory | 语义记忆，保存事实和知识 | | Procedural Memory | 流程记忆，保存做事方法 |

Day06 Memory 会专门展开这一部分。Day04.5 只需要先把名字对齐。

面试表达：

Agent memory is not just chat history. It usually includes short-term conversation context, working memory in runtime state, and long-term memory such as episodic, semantic, or procedural knowledge.

Part G : Tool 系统映射

Tool 是 Day05 的正式主题。Day04.5 先建立术语地基。

我们的说法：

- Tool Runtime
- Tool Definition
- Tool Registry
- Tool Executor
- Tool Result
- Permission Check

工业术语：

| 我们的概念 | 工业术语 | | Tool Definition | Tool Schema / Function Schema | | Tool Registry | Tool Registry / Capability Registry / Tool Catalog | | Tool Executor | Tool Runtime / Action Executor | | Tool Result | Observation | | Permission Check | Guardrail / Approval / Policy Check | | Tool Loop | ReAct Loop / Agent Loop |

Function Calling 与 Tool Calling 的关系：

```
graph TD
    FC[Function Calling] --> TC[Tool Calling]
    TC --> AR[Action Runtime]
```

Function Calling 更偏“模型输出函数调用结构”；Tool Calling 是更大的概念，可以包含：

- Function
- API
- Database
- Browser
- Code Execution
- MCP Tool
- Human Approval

面试表达：

Tool calling is not the model executing APIs directly. The model produces a structured action intent, and the runtime validates, executes, observes, and writes the result back into state.

Part H : Provider Adapter 映射

Day04 Part E 的核心概念是：

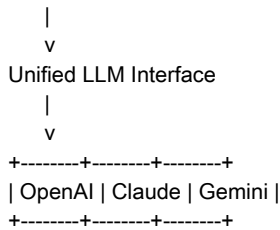
- Provider Adapter

工业界常见叫法：

- LLM Gateway
- Model Router
- Provider Abstraction Layer
- Model Abstraction Layer
- LLM Adapter
- Unified LLM Interface

架构：

- Agent Runtime



现实项目中类似职责的系统：

LiteLLM
OpenRouter
Vercel AI SDK

它们本质都在解决：

- 多模型统一调用
- 供应商协议隔离
- 模型能力差异适配
- 成本与用量统计
- 错误和流事件映射

面试表达：

A Provider Adapter isolates the runtime from vendor-specific model protocols. It converts runtime messages, tools, streaming events, errors, and usage metrics into a unified model interface.

Part I：开源框架对应关系

OpenAI Agents SDK

核心概念：

Agent
Runner
Tool
Guardrail
Tracing
Handoff

映射关系：

| OpenAI Agents SDK | mini-agent-runtime | | Runner | Runtime Loop | | Agent | Agent Definition | | Tool | Tool Runtime | | Guardrail | Permission Check / Policy | | Tracing | Runtime Observability | | Handoff | Agent Orchestration |

LangGraph

核心概念：

StateGraph
Node
Edge
Checkpoint

映射关系：

| LangGraph | mini-agent-runtime | | StateGraph | Runtime State + Workflow | | Node | Runtime Step | | Edge | State Transition | | Checkpoint | State Persistence / Recoverable Snapshot |

Claude Code

Claude Code 更像一个完整 Coding Agent Runtime。

对应我们的：

- Workspace Index
- Context Builder
- Tool Runtime
- Runtime Loop
- Provider Adapter

典型工具：

- Read File
- Edit File
- Bash
- Search

典型上下文：

- 代码结构
- 文件内容
- 执行结果
- Git diff
- 当前计划

MCP

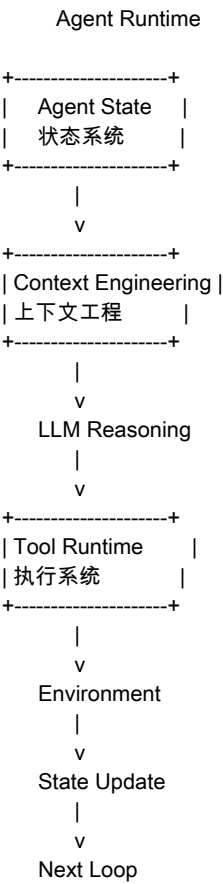
MCP 主要连接 Tool、Resource 和 Prompt。

映射关系：

| MCP | mini-agent-runtime | | Tool | Tool Definition / Tool Execution | | Resource | External Context Source
| | Prompt | Reusable Context Template | | Server | Tool Provider / Capability Provider |

最终统一认知图

重新看 Agent Runtime：



工业统一语言：

Agent State
Context Engineering
Reasoning
Tool Calling
Observation
State Transition
Agent Loop

Day04.5 核心认知升级

1. 不要从 API 开始理解 Agent 框架

以前看框架容易变成：

学习 API
记住调用方法
照着示例拼 Demo

现在应该先找：

State
Context
LLM
Tool
Loop
Persistence
Observability

API 只是这些 Runtime 抽象的外在形状。

2. 工业术语不是新知识，而是已有认知的命名

Day01-Day04 并没有白学，也不需要推翻重学。它们建立的是底层 Runtime 思维。

Day04.5 做的是：

底层认知
+
工业命名
+
框架对应

3. 后续每一天都要保留工业映射结构

从 Day05 开始，每个 Part 尽量保留：

正文学习
|
v
工业术语映射
|
v
框架对应
|
v
面试视角
|
v
思考题

这会让最终笔记更接近：

《从零实现 Agent Runtime：从原理到工业实践》

Day05 前置计划

Day04.5 完成后，正式进入：

Day05：Tool Calling (Execution Engine)

Day05 Part A 先学习 Tool Calling 基础模型：

Tool Calling 为什么让 LLM 从聊天模型变成 Agent

Tool / Function / Action 的区别

Tool Calling 在 Agent Loop 中的位置

ReAct 与 Tool Calling 的关系

OpenAI Agents SDK / Claude Code / LangGraph 中 Tool 的真实定位

mini-agent-runtime 中 Tool 数据模型设计

Day05 整体计划：

Part A：Tool Calling 基础模型

Part B：LLM 如何决定调用 Tool

Part C：Tool Schema 设计

Part D：Tool Registry

Part E：Tool Executor

Part F：Permission & Human Approval

Part G：Tool Result 回流 Runtime

Part H：Multi Tool Loop

Part I：Mini Tool Runtime 实现

本章思考题

为什么说 Agent Runtime 比 Agent API 更重要？

Runtime Loop、Agent Loop、ReAct Loop 三者是什么关系？

Context Builder 和 Context Engineering Pipeline 有什么区别？

Tool Calling 为什么不等于 LLM 直接调用 API？

看一个新的 Agent 框架时，应该优先找哪些 Runtime 抽象？

Source

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a62c303-20d8-83e8-b17a-8f889d4cb725>

- 本地源记录：source/day04-5-chatgpt-share-source.md